Algorithms and Data Structures 2.

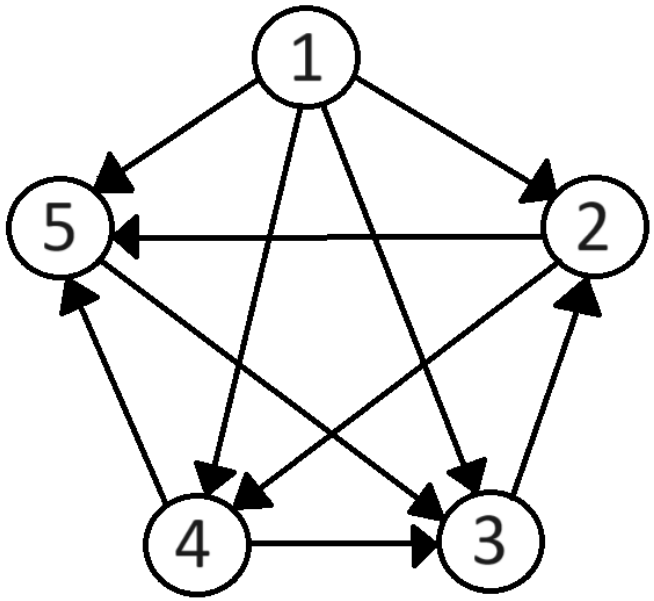
–

Breadth First Search – BFS

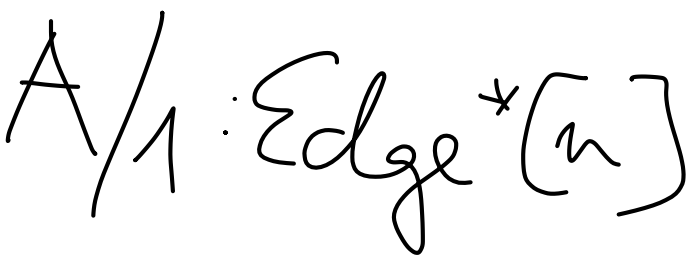
Harrach Nóra - harrachnora@inf.elte.hu

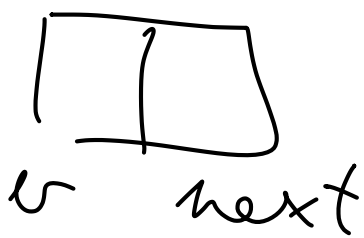
Practice 2

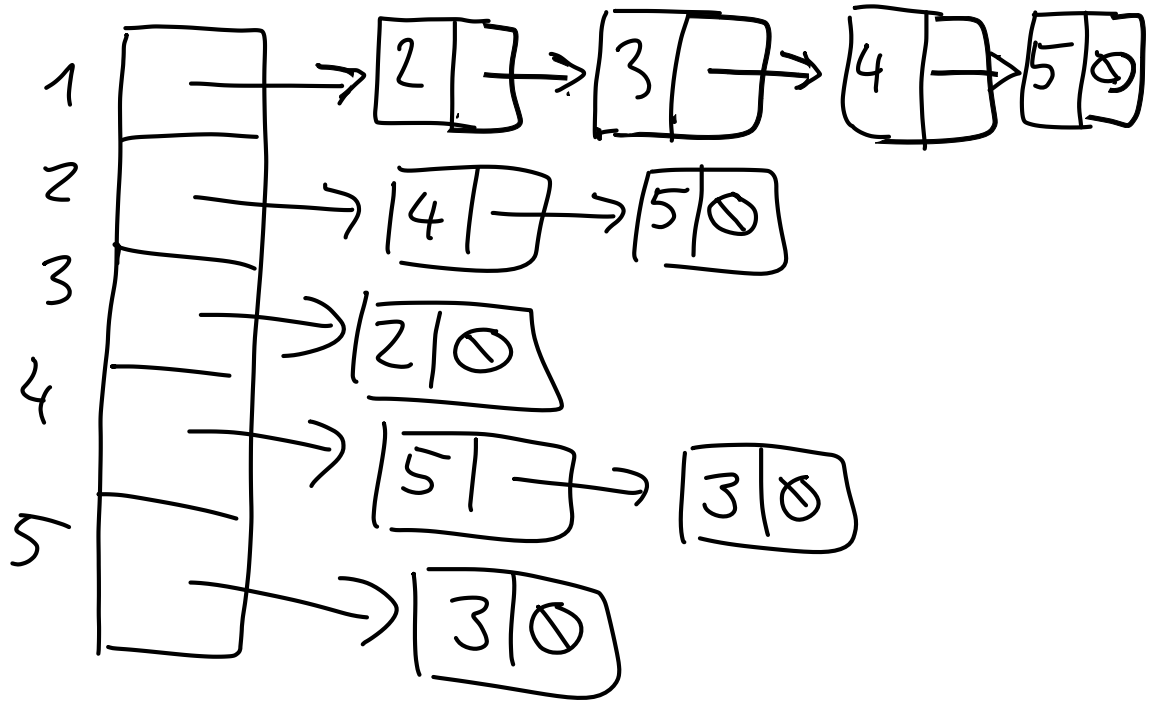
Graph representations



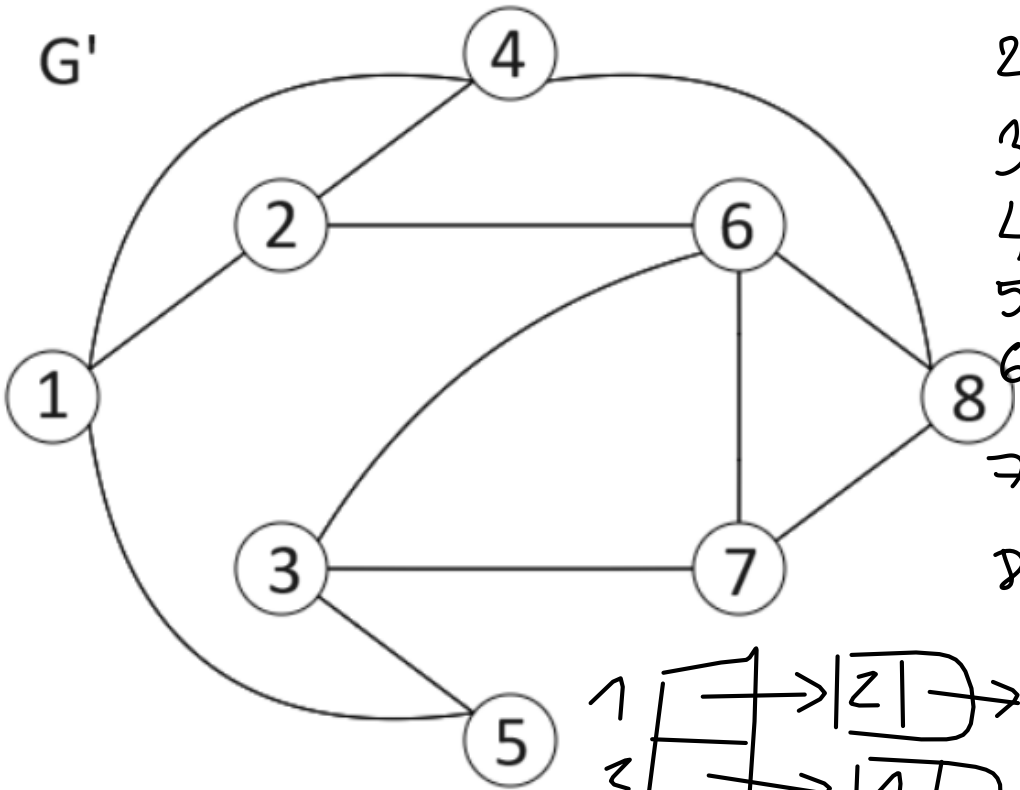
	1	2	3	4	5
1	0	1	1	1	1
2	0	0	0	1	1
3	0	1	0	0	0
4	0	0	1	0	1
5	0	0	1	0	0



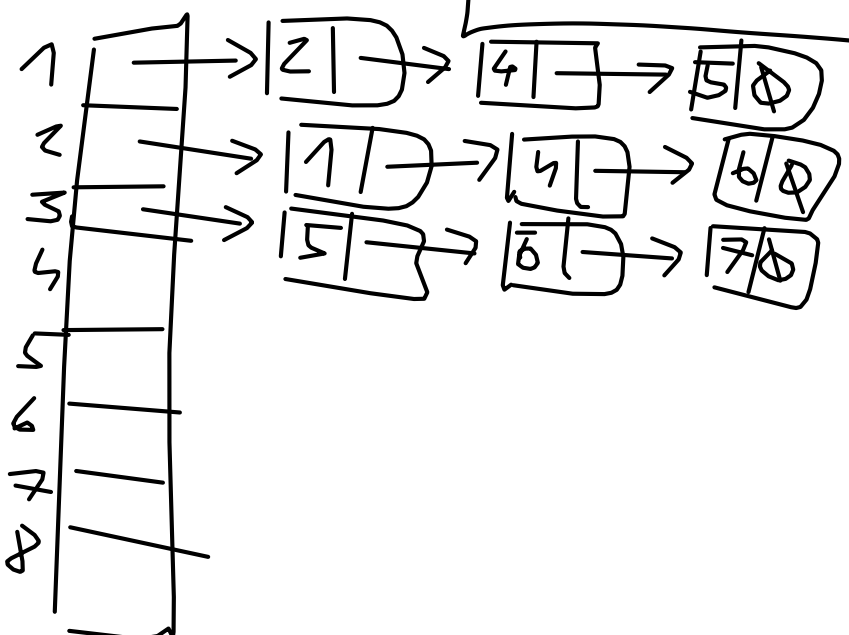




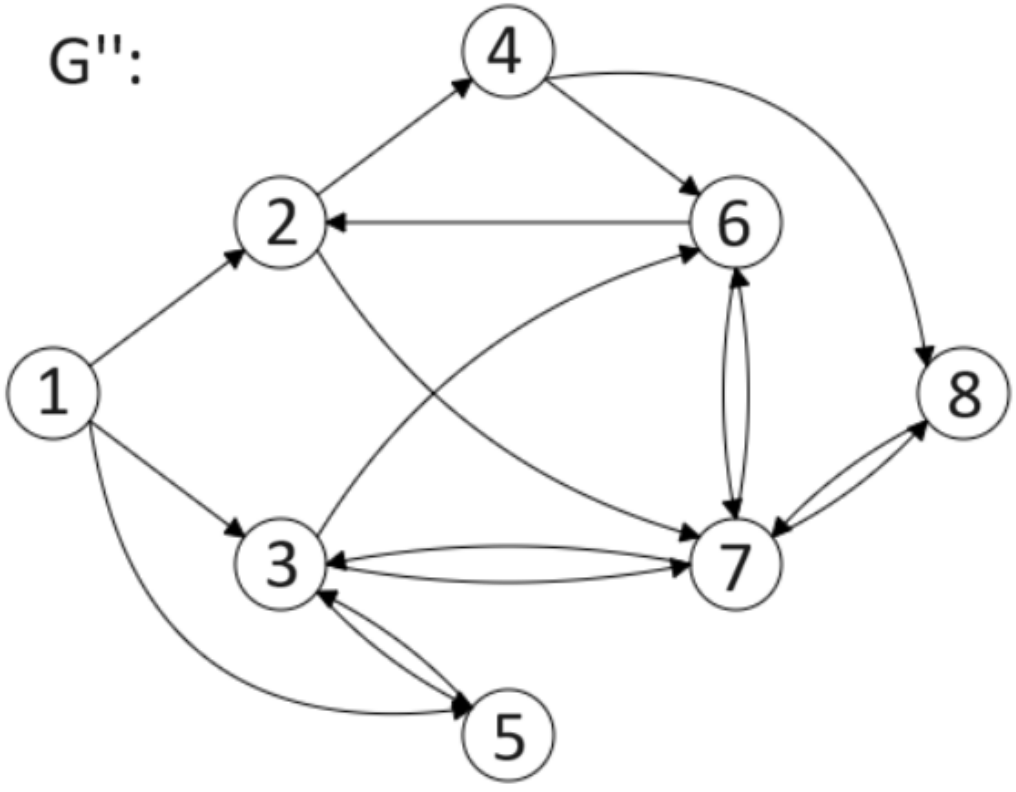
Graph representations



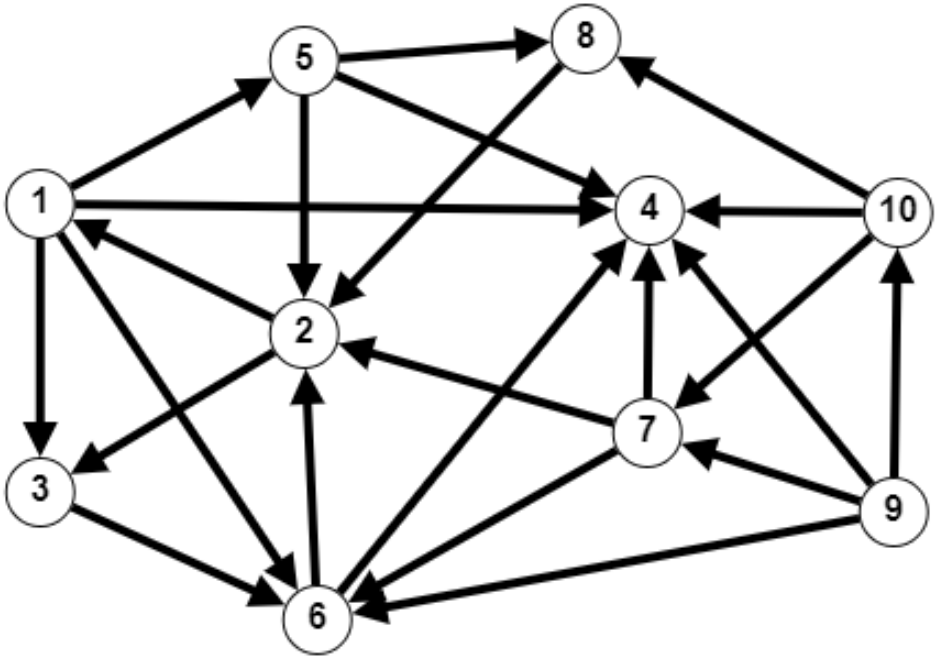
	1	2	3	4	5	6	7	8
1	0	1	0	1	1	0	0	0
2	1	0	0	1	0	1	0	0
3	0	0	0	0	1	1	1	0
4	1	1	0	0	0	0	0	1
5	1	0	1	0	0	0	0	0
6	0	1	1	0	0	0	1	1
7	0	0	1	0	0	1	0	1
8	0	0	0	1	0	1	1	0



Graph representations

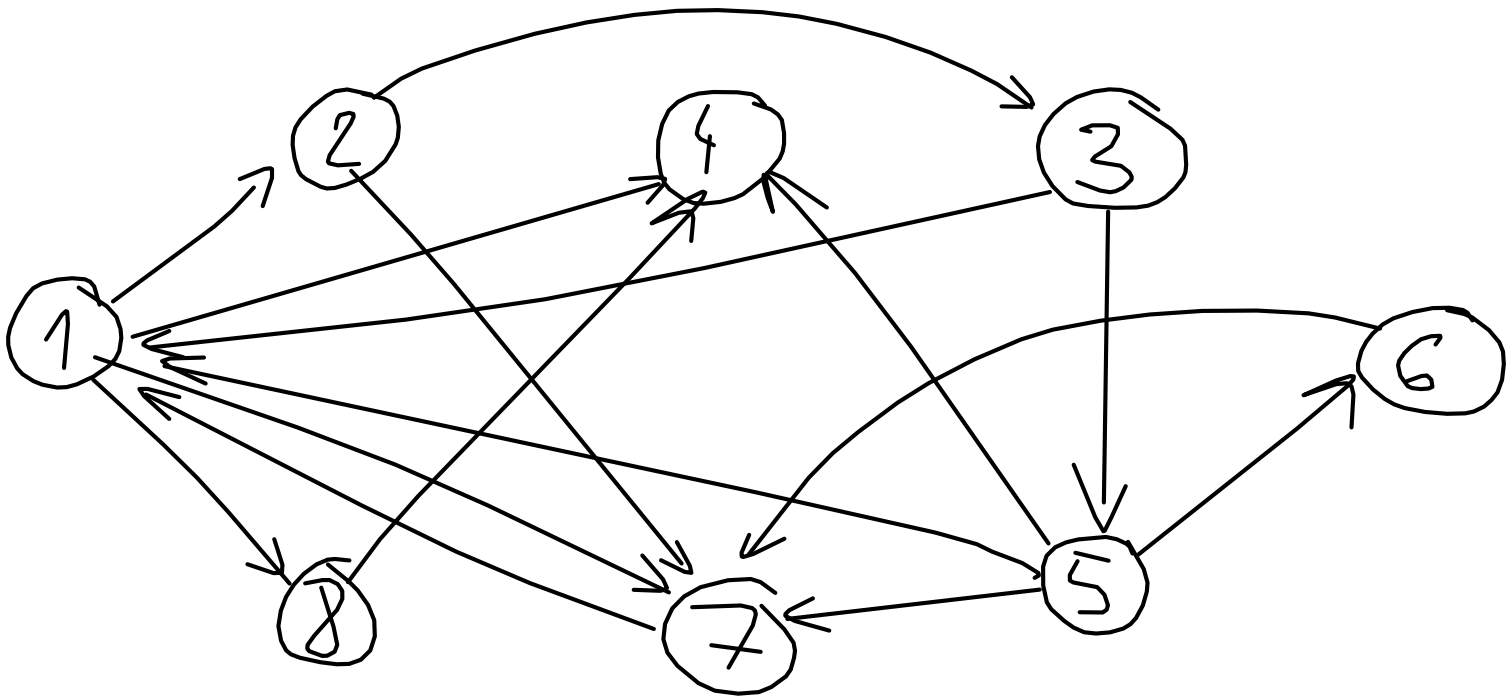


Graph representations



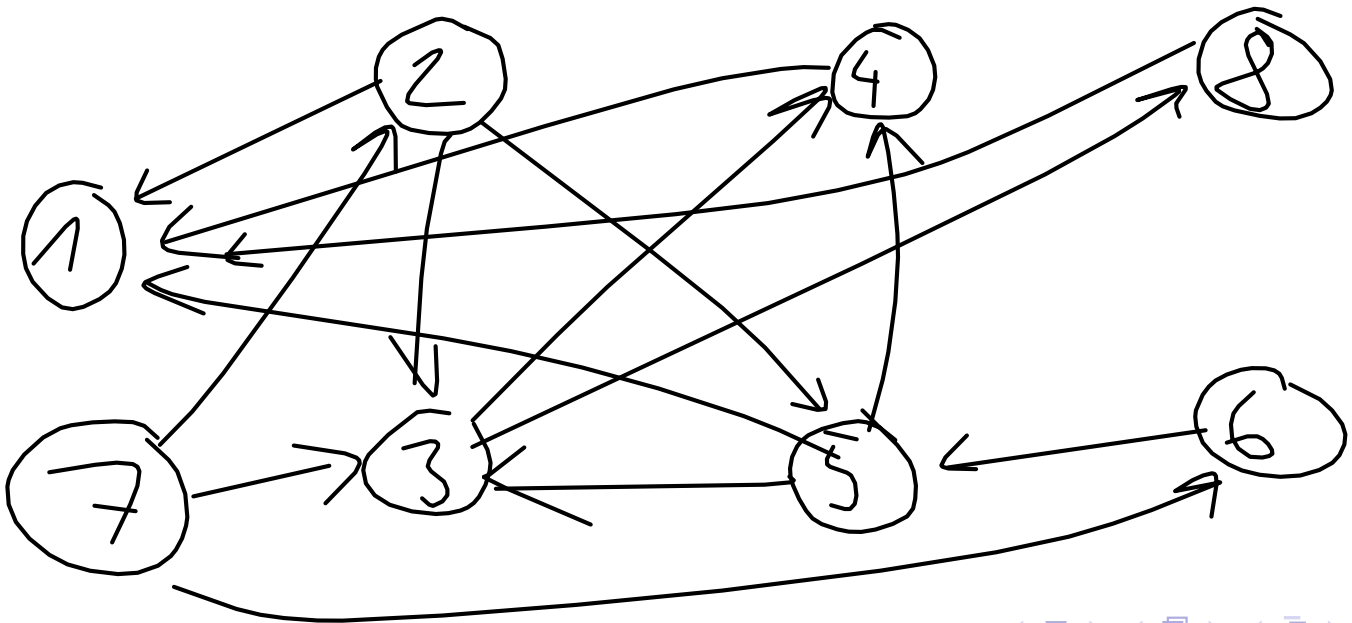
Graph representations

1	$\rightarrow$	2; 4; 7; 8	5	$\rightarrow$	1; 4; 6; 7
2	$\rightarrow$	3; 7	6	$\rightarrow$	7
3	$\rightarrow$	1; 5	7	$\rightarrow$	1
4			8	$\rightarrow$	4



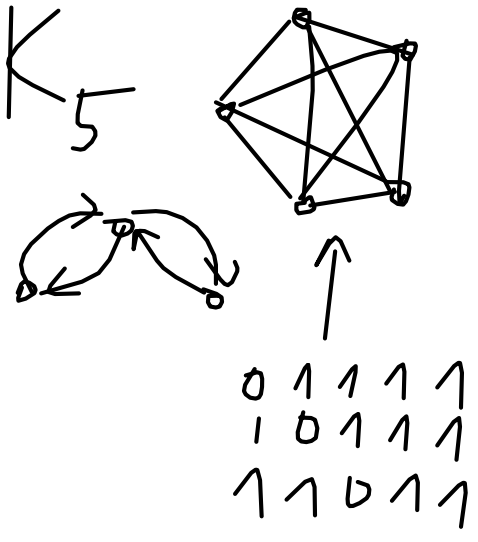
Graph representations

$$A = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$



Exercises

1. Design an algorithm which produces the (a) adjacency matrix, (b) adjacency list representation of the complete graph K_n for given n.

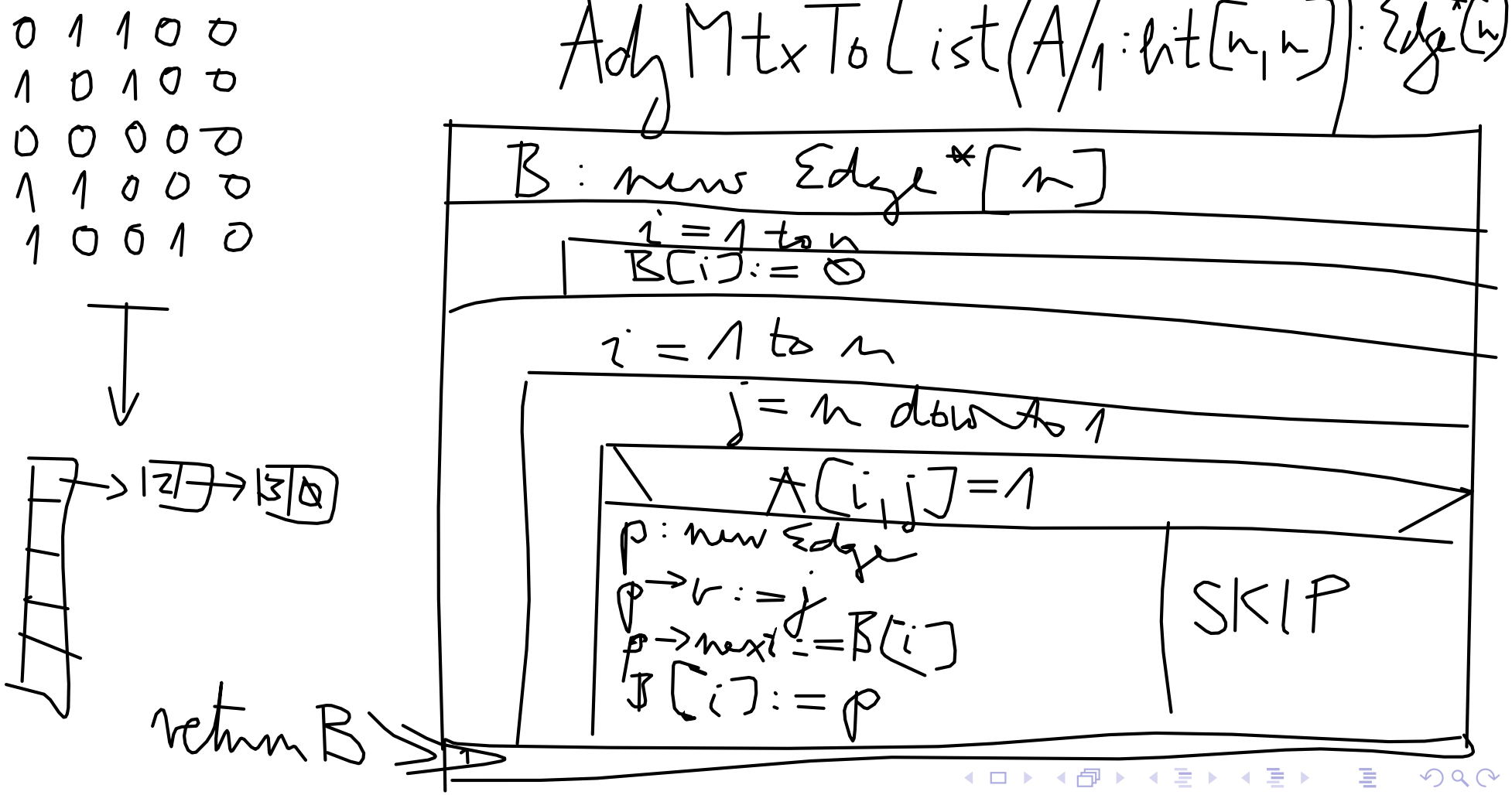



```
Knadymtx(n : ℕ) : int[n,n]
A : new int[n,n]
i = 1 to n
    j = 1 to n
        i = j
        A[i,j] := 0 | A[i,j] := 1
return A
```

Exercises

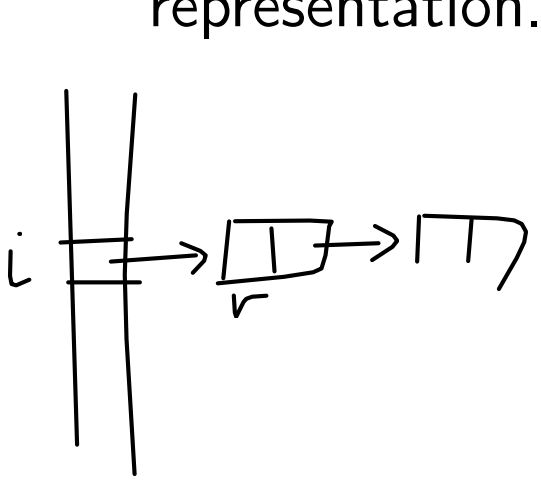
$\Theta(n^2)$

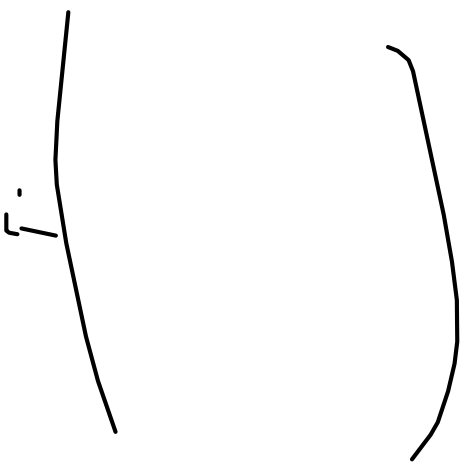
2. Design an algorithm which produces the adjacency list representation of a graph that is given in adjacency matrix representation.

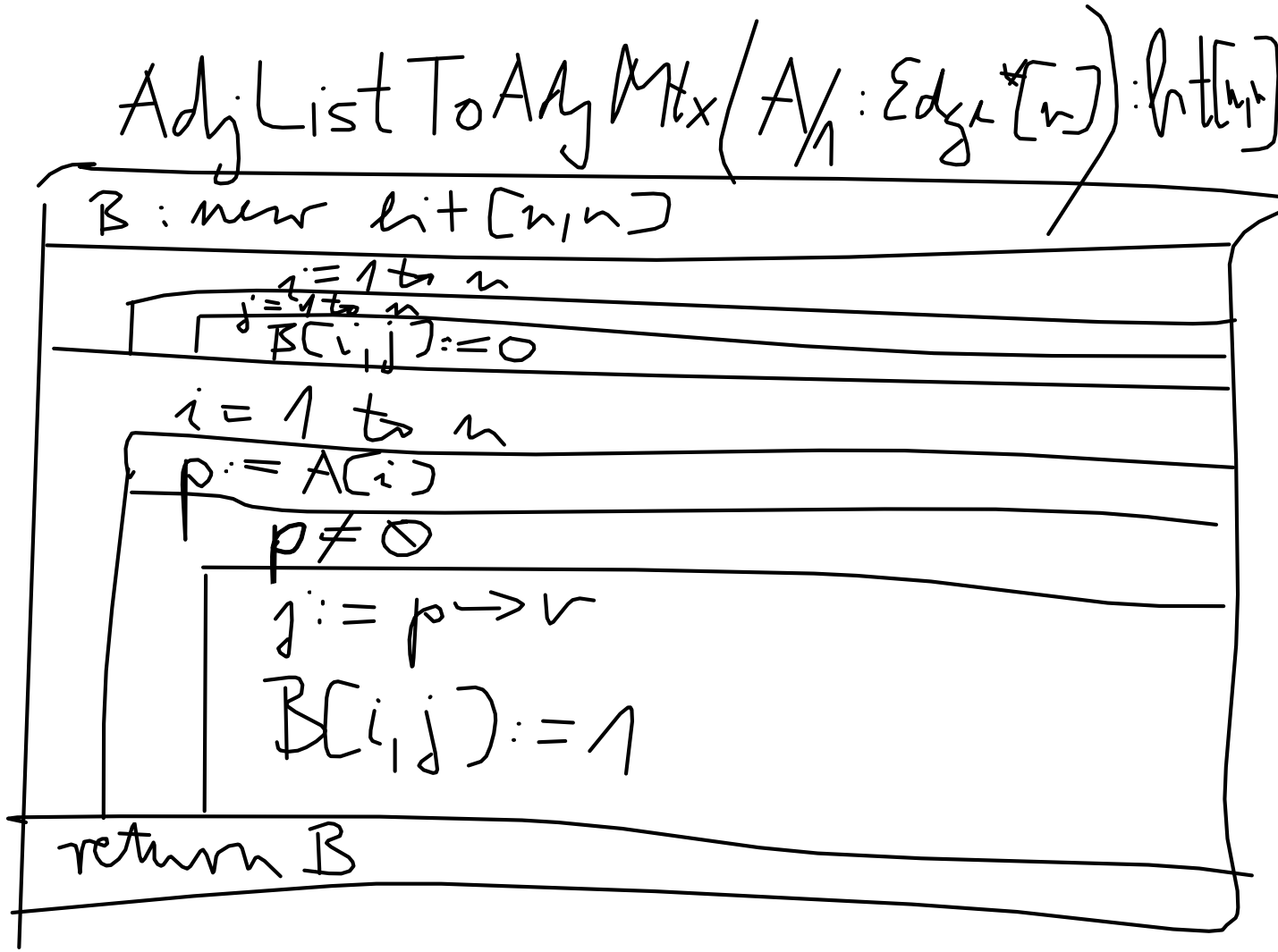


Exercises

3. Design an algorithm which produces the adjacency matrix representation of a graph that is given in adjacency list representation.





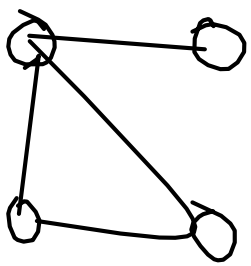


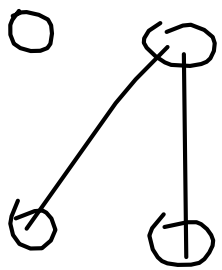
Exercises

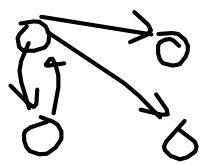
Definition. The **complement** of the graph $G = (V, E)$ is the graph $G' = (V, E')$ with the rule

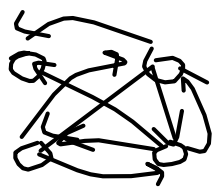
$$(u, v) \in E' \iff (u, v) \notin E$$

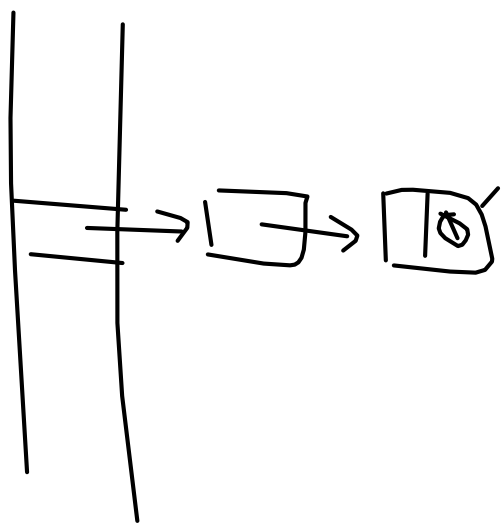
4. Design an algorithm which produces the adjacency list representation of the complement of the graph G which is given with adjacency list representation. We may suppose that all the S1L lists are sorted according to the v values.

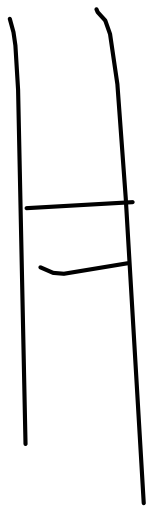


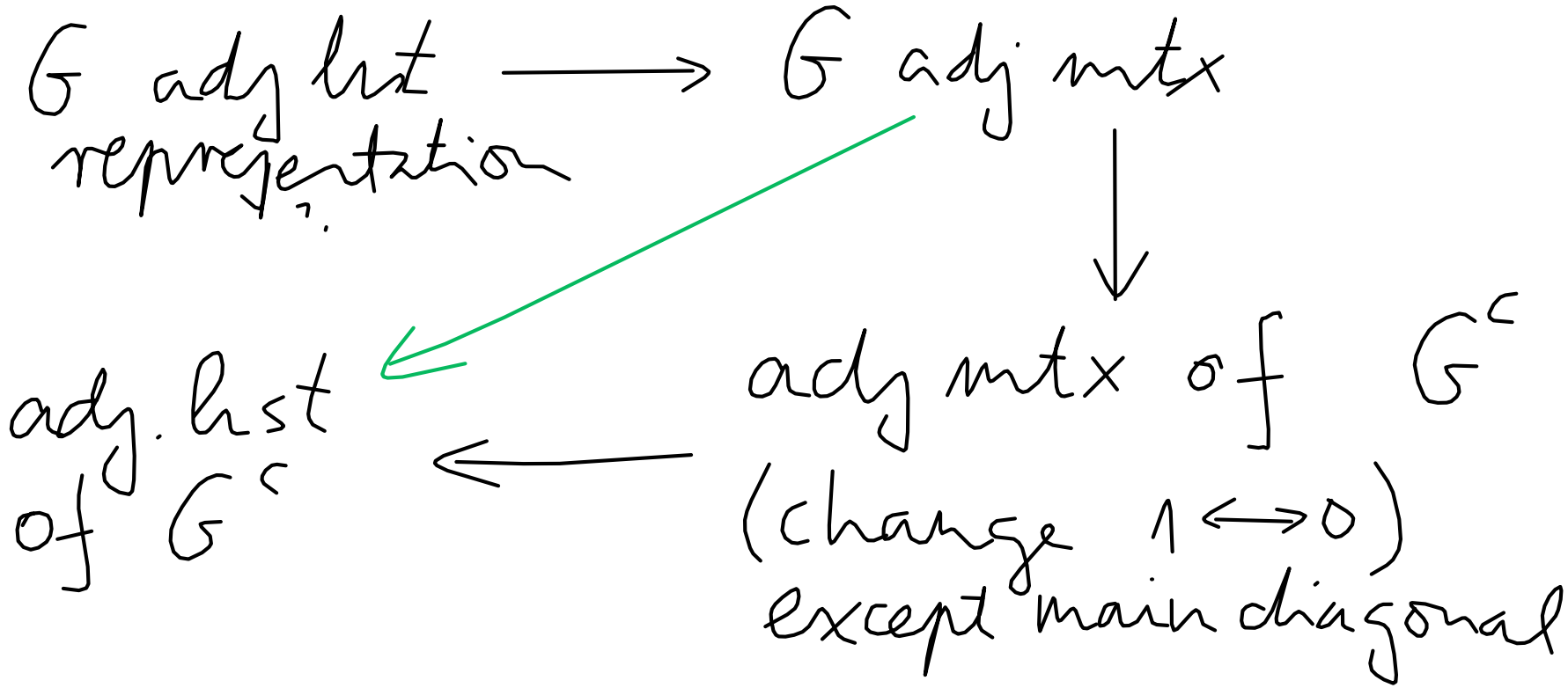












+ idea: we don't need the matrix, only the i^{th} row

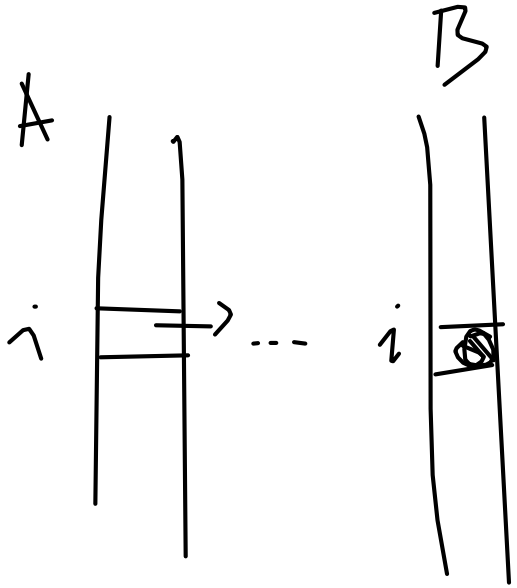
→ use an array for this

Complement(A/1:Edge*[n], B/1: Edge*[n])

```
i := 1 to n
    B[i]:=⊘
    C/1: bit[n] ; C[1,...,n] := [0,...,0]
    p:=A[i]
    p ≠ ⊘
        C[p->v]:=1
        p:=p->next
    C[i]:=1
    j := n downto 1
        C[j] = 0
        p:=new Edge          | skip
        p->v:=j              |
        p->next:= B[i]       |
        B[i]:=p              |
```

array of bits

"main diagonal"



Optional homework

Definition. The **transpose** (or **converse** or **reverse**) of a directed graph G is another directed graph on the same set of vertices with all of the edges reversed compared to the orientation of the corresponding edges in G.

Mathematically: the transpose of the graph $G = (V, E)$ is the graph $G' = (V, E')$ with the rule

$$(u, v) \in E' \iff (v, u) \in E$$

Optional homework. Design an algorithm which produces the adjacency list representation of the transpose of graph G which is given with adjacency list representation. We may suppose that all the S1L lists are sorted according to the v values.

Abstract Graph type

In graph algorithms the graphs are sometimes given as

- $A/1 : bit[n, n] \longrightarrow$ adjacency matrix representation
- $Adj/1 : Edge^*[n] \longrightarrow$ adjacency list representation
- in an abstract way (not specifying which representation): $G : \mathcal{G}$

$\mathcal{E}$
$+\ u, v : \mathcal{V}$

$\mathcal{G}$
$+\ V : \mathcal{V}\{\}$ // vertices
$+\ E : \mathcal{E}\{\}$ // $E \subseteq V \times V \setminus \{(u,u) : u \in V\}$ // edges
$+\ A : V \to 2^V$ // $A(u) = \{\, v \in V \mid (u,v) \in E \,\}$
// $A(u) =$ the adjacent vertices of vertex u.

Then $G.V$ denotes the set of vertices, $G.E$ the set of edges, and $G.A(u)$ the set of neighbours or children of the vertex u.

Notation

Notation. Given two vertices u and v in a graph we will use notation $u \rightsquigarrow v$ if there is a path from u to v.

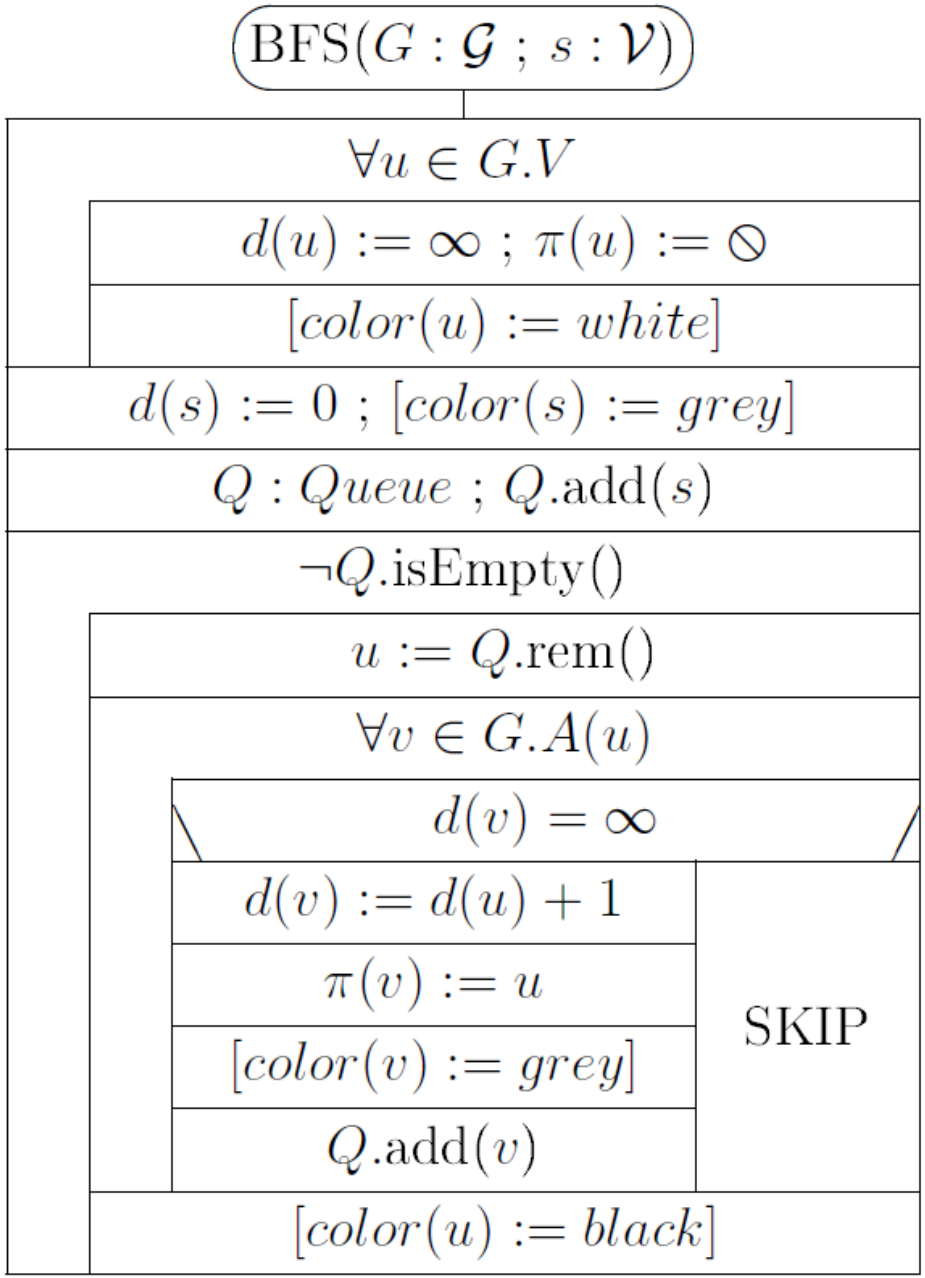
Definition. We say that vertex v is available from u if there is a path from u to v, i.e. $u \rightsquigarrow v$ holds.

Definition. The **distance** of vertex v from vertex u will be the length of the shortest $u \rightsquigarrow v$ path.

Breadth-first search

- Breadth-first search (or BFS) will be considered on both undirected or directed graphs.
- We will fix a source vertex $s \in G.V$.
- We will find the shortest path to each other vertex available from s.
- If there are more than one shortest paths, then we only compute one of them.

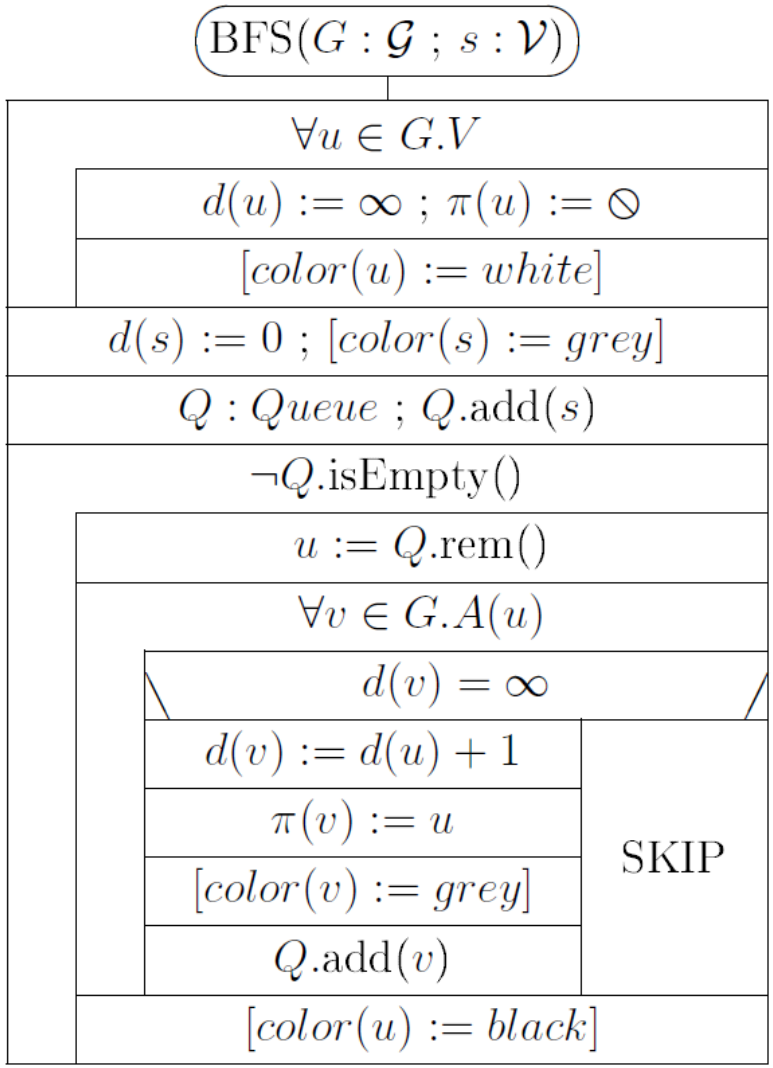
Breadth-first search



Breadth-first search

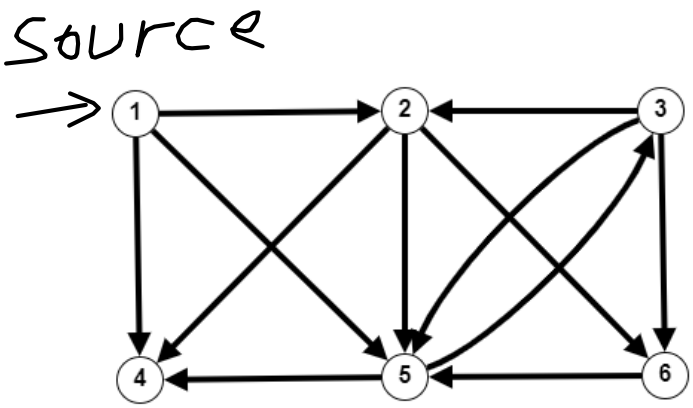
- For every vertex u we will have three labels:
  - $d(u)$ the length of the shortest $s \rightsquigarrow u$ path found
    $d(u) = \infty$ means no such path has been found so far
    $d(u) = 0$ is only true for $u = s$
  - $\pi(u)$ the parent vertex of u on the $s \rightsquigarrow u$ path found.
    $\pi(u) = \oslash$ means that we have not found such path or $u = s$
  - $color[u]$ can be omitted from the program (useful for illustration)
    white if it has not been found yet
    grey if it has been found but not yet processed
    black if it has already been processed
- In the first loop of the algorithm these labels are set:
  $\forall u \in G.V$: $d(u) = \infty$, $\pi(u) = \oslash$ and $color(u) = white$
  for s source: $d(s) = 0$, $\pi(s) = \oslash$, $color(s) = grey$

Breadth-first search



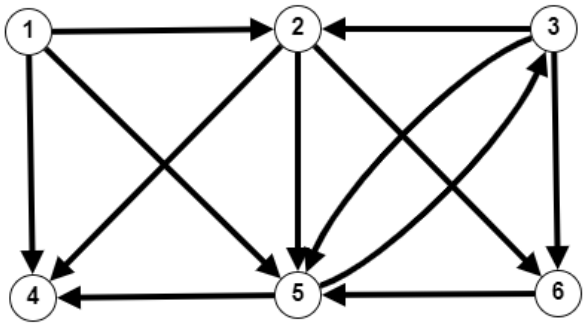
- s is added to queue Q
- In every iteration a vertex u is removed from Q
- and "expanded", i.e. all its neighbours are considered, and if they have not been found before ($d(v) = \infty$) then their d, π and *color* labels are set, and they are added to Q
- Expanded vertices become black.

Example for BFS



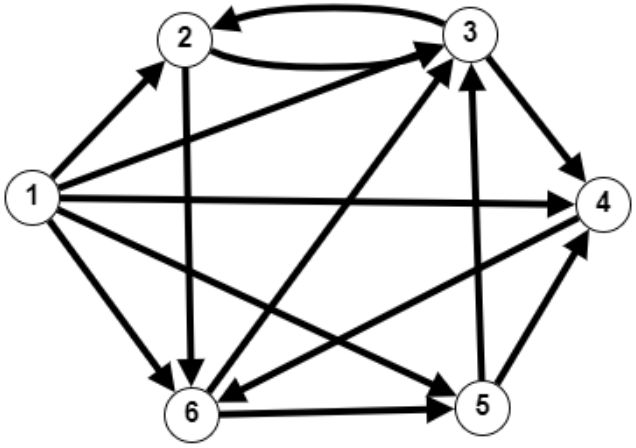
changes of d						ex- panded vertex : d	Q : Queue	changes of π					
1	2	3	4	5	6			1	2	3	4	5	6
0	∞	∞	∞	∞	∞		⟨1⟩	0	0	0	0	0	0
	1		1	1		1	⟨2,4,5⟩		1		1	1	
						final d and π values							

Example for BFS



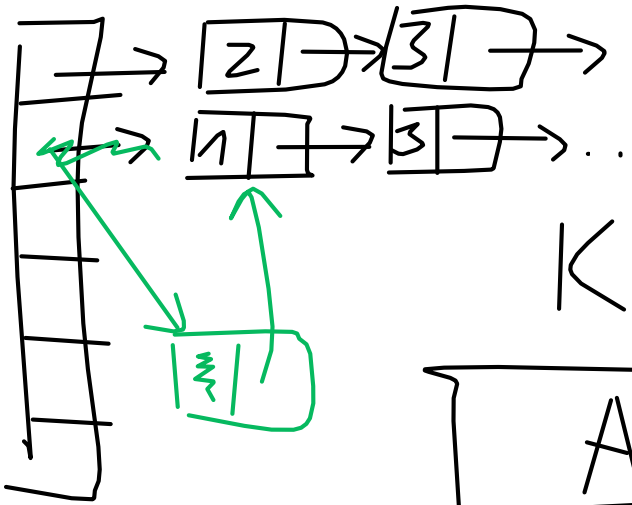
changes of d						ex- panded vertex : d	Q : Queue $\langle \quad \rangle$	changes of π					
1	2	3	4	5	6			1	2	3	4	5	6
						final d and π values							

Example for BFS



changes of d						ex- panded vertex : d	Q : Queue	changes of π					
1	2	3	4	5	6			1	2	3	4	5	6
							$\langle \quad \rangle$						
						final d and π values							

Adj list representation of K_n




```
Kn AdjList(n : ℕ) : Edge*[n]
    A : new Edge*[n]
    i = 1 to n
        A[i] := ∅
    i = 1 to n
        j = n downto 1
            i = j
            SKIP
            p := new Edge
            p→v := j
            p→next := A[i]
            A[i] := p
    return A
```

```
p := new Edge*
p→v := j
p→next := A[i]
A[i] := p
```